
Algorithm 1 CESpGEMM Hierarchical Workflow

```
1: Input: Input matrices  $A, B$  in CSR/CSC format
2: Output: Output matrix  $C = A \times B$ 
3:  $\text{Sem}_{\text{prepare}}[2], \text{Sem}_{\text{compute}}[2], \text{Sem}_{\text{merge}}[2] \leftarrow \text{synchronizers}()$ 
4:  $q_{\text{prepare}}, q_{\text{compute}}, q_{\text{merge}} \leftarrow \text{thread\_safe\_queue}()$ 
5:  $q_{\text{prepare}}.\text{enqueue}(\text{blockA}[0..\text{num\_blocksA} - 1])$ 
6: Launch three pipeline workers in parallel
7: // Level-1: SpGEMM Scheduling
8: // Stage 1: Preprocess Worker
9: for each task  $t \in q_{\text{prepare}}$  do
10:    $\text{Sem}_{\text{compute}}[\text{buffer}_{\text{prepare}}].\text{wait\_avail}()$ 
11:    $\text{flopsB}, \text{flops}_t \leftarrow \text{compute\_flops}(\text{blockA}[i], \mathbf{B})$ 
12:    $\text{blockB}_{\text{new}} \leftarrow \text{blockB\_shuffle}(\text{blockB}, \text{flopsB})$ 
13:    $\text{Sem}_{\text{prepare}}[\text{buffer}_{\text{prepare}}].\text{post\_avail}()$ 
14:    $q_{\text{compute}}.\text{enqueue}(t)$ 
15:    $\text{buffer}_{\text{prepare}} \leftarrow \text{buffer}_{\text{prepare}} \oplus 1$ 
16: // Stage 2: Execution Worker
17: for each task  $t \in q_{\text{compute}}$  do
18:    $\text{Sem}_{\text{merge}}[\text{buffer}_{\text{compute}}].\text{wait\_avail}()$ 
19:    $w \leftarrow \text{get\_blockA\_type}(\text{flops}_t)$ 
20:   // To Level-2: Hybrid Compute
21:    $\text{push\_task}(w, t)$ 
22:    $\text{Sem}_{\text{compute}}[\text{buffer}_{\text{compute}}].\text{post\_avail}()$ 
23:    $q_{\text{merge}}.\text{enqueue}(t)$ 
24:    $\text{buffer}_{\text{compute}} \leftarrow \text{buffer}_{\text{compute}} \oplus 1$ 
25: // Stage 3: Merge Worker
26: for each task  $t \in q_{\text{merge}}$  do
27:   // Merge C and output
28:    $C_{\text{merged}} \leftarrow \text{Merge}(\text{blockC}[0..\text{num} - 1])$ 
29:    $\text{WriteCSR}(C_{\text{merged}})$ 
30:    $\text{Sem}_{\text{merge}}[\text{buffer}_{\text{merge}}].\text{post\_avail}()$ 
31:    $\text{buffer}_{\text{merge}} \leftarrow \text{buffer}_{\text{merge}} \oplus 1$ 
```

Algorithm 2 Asynchronize GPU pipeline in Level-2 Workflow

```
1:  $\text{state}_{\text{free}} \leftarrow (\text{TAG}_{\text{H2D}} \mid \text{TAG}_{\text{KERNEL}} \mid \text{TAG}_{\text{D2H}})$ 
2: while ( $\text{blockId} \leftarrow \text{receive\_task}()$ )  $\neq -1$  do
3:   // H2D share data with Kernel
4:   if ( $\text{state}_{\text{free}} \& \text{TAG}_{\text{H2D}} \wedge \neg(\text{state}_{\text{run}} \& \text{TAG}_{\text{KERNEL}})$ ) then
5:      $\text{host\_to\_device}(\text{blockA}[i], \text{blockB}[j], \text{device})$ 
6:   // Kernel share data with other 2 stages
7:   if ( $\text{state}_{\text{free}} \& \text{TAG}_{\text{KERNEL}} \wedge \neg(\text{state}_{\text{run}} \& \text{TAG}_{\text{H2D}} \mid \text{TAG}_{\text{D2H}})$ ) then
8:      $\text{blockB}[j] \leftarrow \text{ASDC\_decompress}(\text{blockB}[j])$ 
9:      $\text{blockC}[i, j] \leftarrow \text{SpGEMM\_kernel}(\text{blockA}[i], \text{blockB}[j])$ 
10:  // D2H share data with Kernel
11:  if ( $\text{state}_{\text{free}} \& \text{TAG}_{\text{D2H}} \wedge \neg(\text{state}_{\text{run}} \& \text{TAG}_{\text{KNL}})$ ) then
12:     $\text{device\_to\_host}(\text{blockC}[i, j])$ 
```
